

Assignment 1: Builder Pattern

Study Report Generator - Markdown and HTML

Course: ShP-2216 - Software Design Patterns (OP 6B06102)

Institution: Astana IT University, School of Computer Engineering

Implementation: Java 17 | Repository owner: unknow9error

1. Introduction

The chosen product is a study report consisting of a title and an ordered collection of sections. A section is either a paragraph with a heading or a bullet list with a heading. Reports are built incrementally and can have different numbers of sections. The result must be available as Markdown for repository documentation and as HTML for viewing in a browser.

Builder fits this problem because it separates the steps used to assemble a report from the syntax used to represent it. The client calls `title()`, `addParagraph()` and `addBulletList()`, then asks `build()` for the completed product. The same steps can produce either representation. This is more useful than a large constructor when the number and ordering of sections vary.

The product is the immutable Report record. Its fields are title, ReportFormat and the serialized content. The two concrete builders produce structurally different documents: Markdown uses heading prefixes and list markers; HTML includes a document declaration, metadata, semantic sections and list elements. They do not merely select different values for an otherwise identical output.

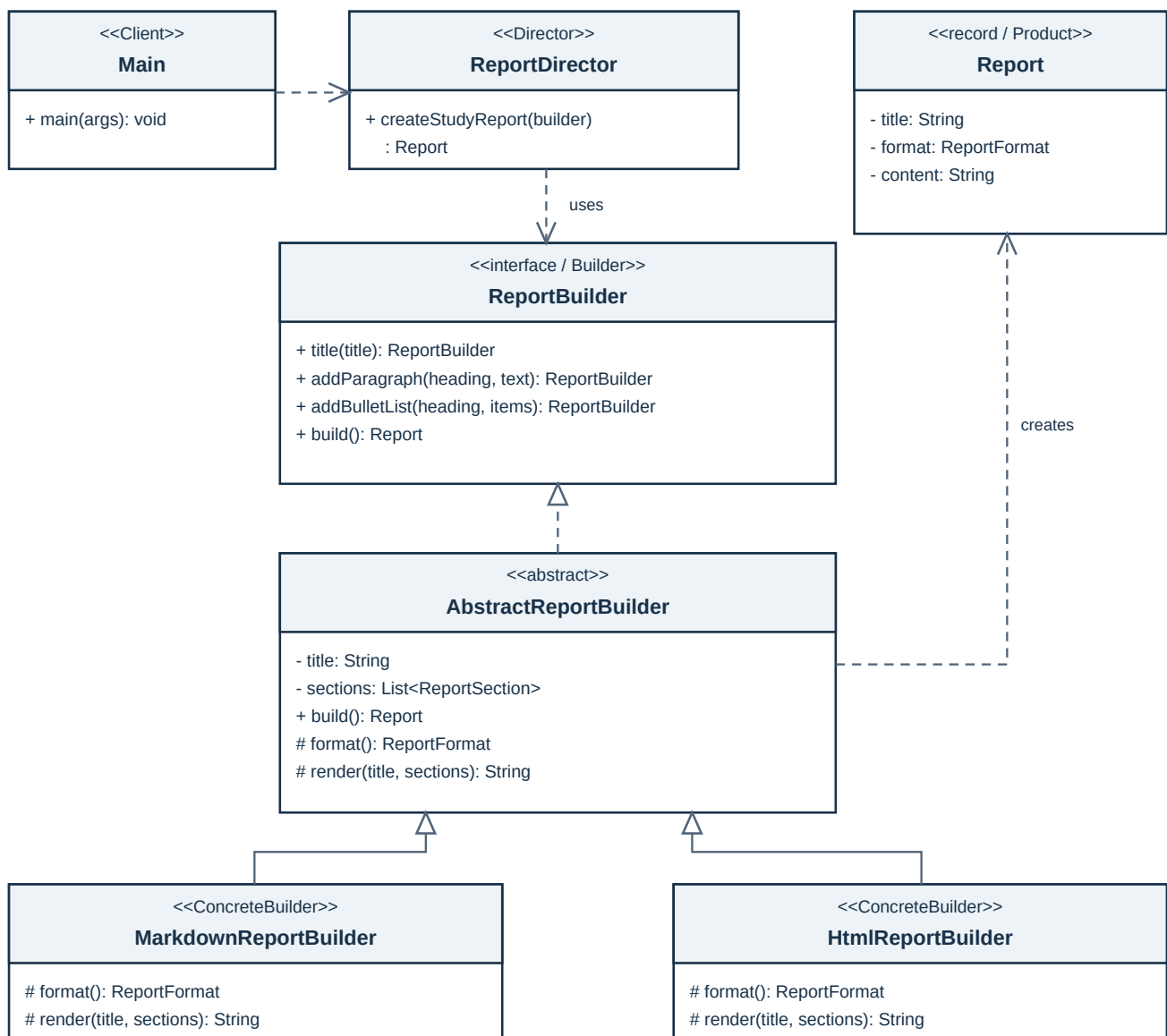
Design decisions

ReportBuilder is the public fluent interface. AbstractReportBuilder is an internal base class that owns construction state, shared validation and final product creation. The concrete builders implement only format selection and serialization. ReportDirector supplies a reusable study-report recipe through the interface. Main selects builders and performs console/file output.

Serialization happens at `build()` after section data has been collected. This keeps validation and construction independent from markup and makes previous products stable when a builder is reused. A Director is useful for repeatable reports, but custom reports can use the fluent API directly.

For a tiny fixed report, a function or constructor would be simpler. The extra types are justified here by variable section construction and two output representations. The scope is deliberately limited to plain text paragraphs and bullet lists.

2. UML class diagram



The diagram shows the main pattern roles. Dashed arrows indicate dependencies; hollow triangles point toward an interface or superclass. The realization from **AbstractReportBuilder** to **ReportBuilder** is dashed, while concrete-builder inheritance is solid. **Main** chooses the two concrete builders, invokes the **Director** and saves the products; its concrete-builder dependencies are listed here to keep the drawing readable.

Supporting types omitted from the diagram are **ReportSection** (an immutable section and its **Kind**), **ReportFormat** (MARKDOWN/HTML with file extensions), and **TextValidation** (shared input checks). The editable PlantUML source in `docs/uml.puml` includes the Client's creation dependencies as well.

Construction flow: **Main** creates a fresh builder; the **Director** invokes fluent steps; the shared `build()` validates completeness; the selected renderer produces content; a new **Report** is returned to **Main**. File writing is a Client responsibility, not a Builder responsibility.

3. Clean Code principles - structure

1. Intention-revealing names

The interface describes domain actions instead of generic setData() operations. A caller can read the construction chain as a sequence of report-writing steps. Source: ReportBuilder.java.

```
ReportBuilder addParagraph(String heading, String text);
ReportBuilder addBulletList(String heading, List<String> items);
Report build();
```

2. Small methods with focused responsibilities

The HTML render() method handles the document shell and iteration. Section serialization is delegated to appendSection(); escaping has its own helper. Main handles output separately, so a formatting change does not require changing file-writing code. Source: HtmlReportBuilder.java, excerpt from render().

```
for (ReportSection section : sections) {
    appendSection(html, section);
}
```

3. Shared construction logic, without duplication

Both concrete builders inherit the same title/section accumulation and final build() implementation. Only format() and render() vary. This prevents one builder from accepting incomplete reports while the other rejects them. The final build method centralizes product creation. Source: AbstractReportBuilder.java, final statement of build().

```
return new Report(title, format(),
    render(title, List.copyOf(sections)));
```

The base class is package-private because its render contract uses the internal ReportSection type. Code outside the package depends on the public ReportBuilder interface and the two concrete builders. A new representation can share the base inside the package or implement the interface independently.

3. Clean Code principles - correctness

4. Validate construction with clear failures

Invalid field input is rejected when a step is called. The final build() rejects incomplete state, giving the caller a specific correction. Source: AbstractReportBuilder.java.

```
if (title == null) {
    throw new IllegalStateException(
        "Report requires a title before build()");
}
```

5. Encapsulate mutable state

Report contains only immutable strings and an enum. ReportSection collects validated strings into a new unmodifiable list, preventing later changes to the caller's list from changing a report. Source: ReportSection.java.

```
items = items.stream()
    .map(item -> TextValidation.requireSingleLine(
        item, "Section item"))
    .toList();
```

6. Replace format strings with a named type

ReportFormat defines the supported formats and extensions in one place. Main asks the product for its extension instead of repeating string comparisons. Source: ReportFormat.java and Main.java.

```
MARKDOWN("md"),
HTML("html");
// Excerpt from Main.writeReport():
Path destination = directory.resolve(
    name + "." + report.format().extension());
```

7. Comments explain non-obvious intent

The Markdown escaping helper explains why punctuation is escaped; it does not narrate the replaceAll() call. Other routine steps rely on names rather than redundant comments. Source: MarkdownReportBuilder.java.

```
// CommonMark backslash escapes keep user-supplied punctuation literal.
return text.replaceAll("([\\p{Punct}]]", "\\$1");
```

4. Verification and usage

The implementation was compiled and executed with Amazon Corretto 17.0.13. Compilation uses `--release 17`, UTF-8 encoding, all javac warnings and warnings-as-errors. The dependency-free runner passed 43 tests. The demo generated both `output/study-report.md` and `output/study-report.html`, plus a direct fluent-API example in the console.

Tests cover missing titles/sections, null or blank input, multiline fields, empty and invalid lists, fluent builder identity, caller-list mutation, stable previously built products, repeated `build()`, failed-step state preservation, Unicode, whitespace normalization, exact format output, escaping and a common Director recipe across both builders.

```
bash scripts/test.sh
bash scripts/run.sh
```

The GitHub Actions workflow repeats compilation, tests and the demo on Java 17 for pushes and pull requests. The local results above are separate from the status of any remote workflow execution.

5. Conclusion: benefits and tradeoffs

The implementation provides one readable construction process for two useful representations. The most valuable separation is between assembling report sections and rendering markup. Shared validation and immutable products make failures predictable and prevent accidental changes to previously built results. The Director demonstrates reuse, while direct fluent construction preserves flexibility.

A concrete issue encountered during review was that leading spaces in a Markdown paragraph could create a code block. Stripping surrounding whitespace at the field boundary fixes that behavior consistently for both formats, and a regression test records the intended contract.

The design also has costs. It uses more classes than a single formatting function, holds the full report in memory and requires a new enum value when adding a format. Input fields are single-line plain text; nested lists, images, raw markup and streaming output are unsupported. Builders are mutable and not thread-safe. `build()` intentionally does not reset state, so each Director recipe receives a fresh builder.

These tradeoffs suit a small teaching project. For large documents, streaming and richer section types would require additional design work. Personal defense preparation should focus on why these boundaries were chosen and when a simpler constructor or formatter would be preferable.

6. Repository and materials

GitHub: [unknown9error/aitu-software-design-pattern-assignment1](https://github.com/unknown9error/aitu-software-design-pattern-assignment1)

The supplied Assignment 1 instructions define the requirements. Source, tests, UML source, example output and this report are included in the repository. The Java application needs no third-party dependencies; ReportLab is used only to generate this PDF from the Markdown report.